

System Architecture

River Hawk Consulting, LLC · IMAX HCD Prototype OT
August 8, 2026

CONTENTS

System Architecture

1. What the system is

Why six and not one

2. Components

2.1 imax-spa — SPA host and API edge

2.2 The five enrichment services

2.3 The client

3. Certificates and TLS

4. Identity and authorization

5. State, data and persistence

6. Production cutover

6.1 Which model answers which call

6.2 What this depends on from the gateway — and what it deliberately does not

6.2.1 Parameter divergence within chat completions is negotiated, not configured

6.3 What the gateway must return

7. Deployment

8. Interfaces

8.1 External

8.2 API surface

8.3 Configuration

9. Rotation, scaling and failure

10. What the Government still has to supply

System Architecture

Agreement No. 5600002690012 · River Hawk Consulting, LLC · UNCLASSIFIED IMAX Chat-Viewer Prototype

Delivered under **Article XII, Data Rights**: *"the architecture the Performer will deliver prior to the end of the period of performance."*

Article I(b) defines what that word has to mean here:

"The overall design and structure of the system, including the relationships between its components, interfaces, and data flows... sufficient to enable the Government to implement the system without requiring further development or support from the Performer."

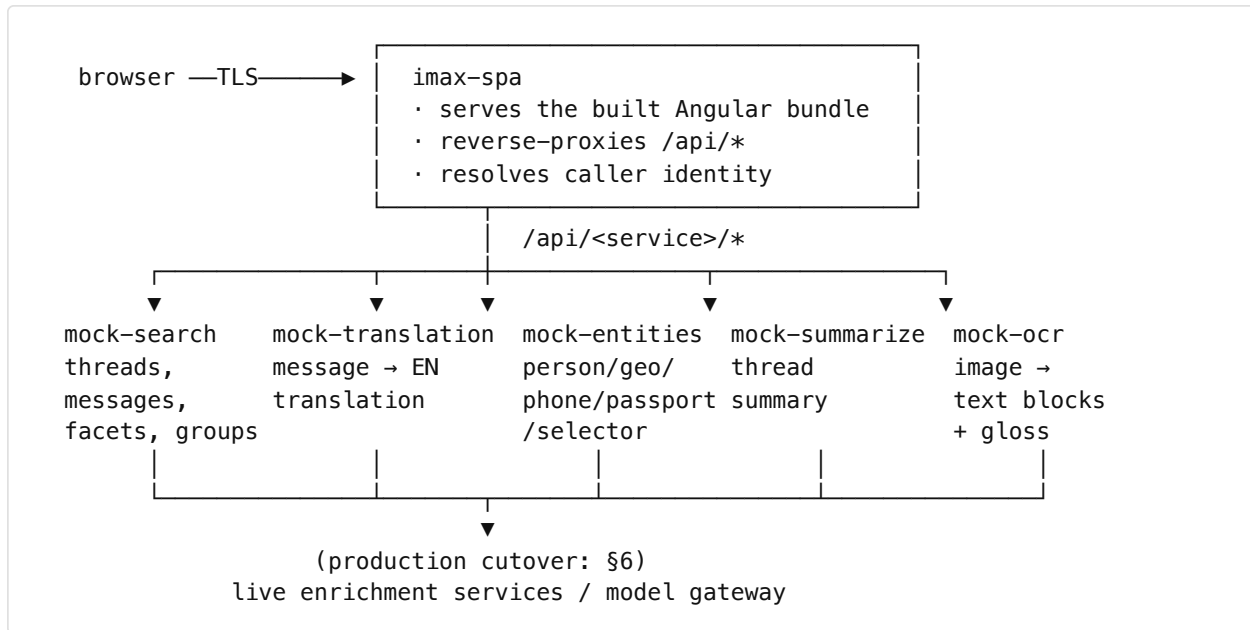
That last clause sets the bar. This document is written to it: it describes not only how the prototype is put together but everything the Government needs in order to stand it up, change it, and cut it over to production services without asking River Hawk anything. Where something genuinely cannot be known outside the enclave, it says so and names the value that has to be supplied.

Companion documents: `developer_guide.md` (how to work in the codebase day to day), `deploy/README.md` (deployment procedure), `deploy/AIRGAP.md` (restricted-network transfer). This document is the one that has to stand alone.

1. What the system is

A single-page application for linguists and targeting officers triaging multi-language intercepted communications, plus a service layer that emulates the production enrichment services behind frozen contracts.

Six containers. One serves the SPA and proxies the API surface; five are single-responsibility enrichment services. Nothing shares a database, because nothing has one — see §5.



WHY SIX AND NOT ONE

Each service owns one contract and can be replaced independently. At cutover the Government does not swap “the backend” — it repoints one service at a time and keeps the rest on fixtures until each is proven. The seam is per-service by design, and §6 is the procedure.

2. Components

2.1 IMAX-SPA — SPA HOST AND API EDGE

- **Image:** `deploy/Dockerfile.spa`. Node runtime, non-root, read-only root filesystem.
- **Entrypoint:** `deploy/spa-entry.js`. An Express edge, not nginx — deliberately, so the identity logic and the proxy live in the same reviewable process as the TLS termination.
- **Responsibilities**
 1. Serve the compiled Angular bundle and the fabricated corpus’s static assets.
 2. Reverse-proxy `/api/<service>/*` to the corresponding service, per `UPSTREAM_*`.
 3. Resolve caller identity from what the authenticating front sends, and refuse unauthenticated requests (§4).
 4. Terminate TLS using material fetched at pod start (§3).

2.2 THE FIVE ENRICHMENT SERVICES

All five share `deploy/Dockerfile.mock` and `deploy/mock-entry.js`; the route module differs.

Service	Route module	Contract	Production analogue
mock-search	routes/search.js	threads, thread messages, message search, facet counts, groups	Corpus search / retrieval
mock-translation	routes/translation.js	message → English, with service attribution	Machine translation
mock-entities	routes/entities.js	message → typed entities with confidence	Entity extraction
mock-summarize	routes/summarize.js	thread → executive summary	Summarization
mock-ocr	routes/ocr.js	attachment → text blocks, per page, plus English gloss	OCR / document exploitation

Each answers **schema-pinned JSON**. The schemas are the interface contract, not an implementation detail: the client's typed models mirror them exactly, so a drift surfaces as a compile error rather than a runtime surprise. `tests/services.spec.js` asserts the wire format independently of the UI, which is what makes the contracts testable at cutover.

2.3 THE CLIENT

Angular 21.2.19, pinned exactly. Standalone components, signals, no NgModules.

```
angular/src/app/
  app.ts / app.html      shell: toolbar, UNCLASS banner, three panels
  core/
    models/              typed mirrors of the service contracts
    api/                 one wrapper per service
    stores/              triage · session · gold-copy (signal state)
    services/identity    who the front says is signed in
    directives/panel-resize shared drag-to-resize
  features/
    triage/              Search & Triage – queue, facets, groups, filters
    viewer/              chat stream, message bubble, OCR scan viewer
    gold-copy/           promoted threads and export
```

Three stores, three lifetimes. `TriageStore` holds what is selected. `SessionStore` holds the officer's work — cached translations, review verdicts, notes, document tags. `GoldCopyStore` holds promoted threads and their order. All three are in-memory signals with no persistence layer, which is a requirement rather than an omission (§5).

3. Certificates and TLS

Serving TLS requires a certificate and key. In the target environment these live in AWS Secrets Manager behind a role, so the cluster fetches them itself rather than a human staging a Kubernetes Secret.

Flow. An initContainer (`deploy/secrets-init.js`) runs before the application container, authenticates, fetches the material, validates it, and writes `tls.crt` / `tls.key` / `ca.pem` mode `0600` into an in-memory `emptyDir`. The application container mounts the same volume at `/etc/tls` and reads it synchronously at import (`deploy/tls.js`). The private key never becomes a Kubernetes Secret and never touches disk.

Credential chain, in order, so both credential models work with the same manifests:

- 1. `AWS_WEB_IDENTITY_TOKEN_FILE` + `AWS_ROLE_ARN` → `AssumeRoleWithWebIdentity` (IRSA)
- 2. `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` / `AWS_SESSION_TOKEN`
- 3. IMDSv2 → instance profile

Secret layouts handled without a code change, because the layout was not knowable in advance: two secrets (cert and key separately); one secret carrying both; raw PEM; JSON under any of several key spellings; and base64 inside JSON. Anything else is refused by name, loudly, before the pod starts — a wrong secret fails here rather than three layers down as a TLS handshake error.

Signing is SigV4 over `node:crypto` (`deploy/aws-sigv4.js`), with no AWS SDK. The SDK would have added roughly forty transitive packages that the enclave’s npm mirror has to carry and the vulnerability gate has to clear, for one signed POST.

What the Government must supply at deploy time — none of it is in the repository:

Value	Where	Why absent
<code>AWS_SECRETSMANAGER_ENDPOINT</code>	<code>imax-tls-source</code> ConfigMap	Regional endpoint; committing enclave hostnames is blocked by <code>scripts/preflight-airgap.sh</code>
<code>TLS_CERT_SECRET</code> , <code>TLS_KEY_SECRET</code> , <code>TLS_CA_SECRET</code>	same	Secret names are unknown outside the account
Role ARN	<code>deploy/k8s/serviceaccount.yaml</code> annotation	Unset falls through to the node role

Overlay: `deploy/overlays/tls-awssm`. The file-based overlay `deploy/overlays/tls` is retained for environments with no AWS.

4. Identity and authorization

The prototype does not authenticate anyone. An authenticating front does, and forwards the result; the pod trusts that front, which is the same posture the deployment already takes by having no other route in.

Two modes, selected by `AUTH_MODE`:

- `bearer-jwt` (the target). The front sends the identity token in `Authorization` and the access token in `x-auth-request-access-token`. The pod `base64url-decodes` the payload and reads claims. **The signature is not verified** — deliberately, and logged at startup so the assumption is visible in `kubectl logs` rather than buried in a comment.
- `proxy-header`. Flat headers (`x-ain`, `x-name`, `x-ismemberof`, `x-org`) for any front that sends them.

Both produce the same `{id, name, org, groups, label}`, so everything downstream — group authorization, `/api/whoami`, the toolbar — is mode-agnostic.

Claim names are configuration, not code: `AUTH_CLAIM_ID`, `AUTH_CLAIM_NAME`, `AUTH_CLAIM_GROUPS`, `AUTH_CLAIM_ORG`. The defaults (`sub`, `name`, `groups`, `org`) are the common OIDC spellings and are **not confirmed** against the Sponsor's STS. `/api/whoami` reports which claim names a real token actually carries — names only, never values — so a mismatch is diagnosed in one request and corrected with a ConfigMap edit.

`AUTH_REQUIRED_GROUPS` gates access on group membership. Empty means any authenticated caller.

5. State, data and persistence

The system stores nothing. No database, no cache, no session store, no client-side persistence — no `localStorage`, `sessionStorage`, `IndexedDB` or cookies anywhere in either build. Officer work — translations, verdicts (including whether one was reached line by line or by accepting a run in bulk), per-message notes, the conversation-level note, tags, promoted threads and panel widths — lives in memory for the life of the tab.

This is a deliberate prototype property with two consequences the Government should weigh:

- **Every pod is disposable and horizontally scalable.** Any replica can serve any request.

- **A reload loses the officer's work.** For a prototype demonstrating a workflow this is correct; for production it is the first thing to specify, and the seam is `SessionStore` — a single injectable holding all of it.

The demonstration corpus (`data/seed.js`) is **fabricated, unclassified** demonstration data: 46 threads (largest 2,388 messages) across Arabic, Farsi, Chinese and Russian, three ingest lanes (message, transcript, document), and document scans including a five-page Arabic customs declaration. Every screen carries an UNCLASS banner.

6. Production cutover

The point of the seam. Per service, not all at once:

1. Point `UPSTREAM_<SERVICE>` at the production service.
2. Confirm the response satisfies the pinned schema — `tests/services.spec.js` is the check, and it runs against whatever the variable points at.
3. Repeat. Services still on fixtures keep working; there is no flag day.

Model gateway. The enrichment services call a live gateway when `MODEL_ENDPOINT`, `MODEL_NAME` and `MODEL_API_KEY` are supplied, and fall back to fixtures otherwise. Three things fail in ways that do not look like their cause, so they are stated here:

- `enabled()` requires **both** endpoint and key. Miss either and the pods answer from fixtures — a green deployment quietly not using the gateway. Each pod logs which mode it is in at startup.
- The endpoint must be the base URL **including the version path** (`https://host/v1`); the code appends `/chat/completions`. A URL with the suffix already on it 404s in a way that reads like an outage.
- The host is parsed into the provenance label an officer sees on every translation, so a malformed endpoint corrupts the gold copy's audit trail, not just the request.

`MODEL_ENDPOINT` and `MODEL_NAME` live in a ConfigMap; only `MODEL_API_KEY` is a Secret. A URL is not a credential, and burying it in a Secret means it cannot be diffed or corrected without recreating the whole object.

6.1 WHICH MODEL ANSWERS WHICH CALL

The enclave gateway publishes **GPT-OSS 5.2** and **Claude Sonnet 4.5**. They are not interchangeable across the two workloads, so the model is selected per capability:

Call	Variable	Why
Translation	MODEL_NAME	Text-only work, and the high-volume path — a standing channel is thousands of messages
OCR / document exploitation	MODEL_NAME_OCR	A vision call: the page is sent as an image (MODEL_NAME_VISION is accepted as an alias)

Entity extraction and summarization are fixture-backed only. They expose the same frozen contracts and the same `UPSTREAM_*` seam as the others, so they cut over to a production service by configuration — but they do not call the model gateway, and setting `MODEL_NAME` does not make them live. Only translation and OCR do. Stated here because the opposite assumption is easy to make and would be discovered as “the gateway is configured and entities are still canned”.

All four values — endpoint, key, and both model ids — are supplied by the enclave in the **Secret**. Every pod loads the `imax-model-gateway` ConfigMap and then the Secret of the same name, and Kubernetes applies `envFrom` sources in order, so a key present in both takes the Secret’s value. The ConfigMap is the fallback; nothing has to be edited to move a value into the Secret.

Point OCR at a model that cannot accept an image and every page fails identically. The gateway answers 400, the route falls back to the fixture, and the deployment looks wired while no document is ever actually read. That failure is now visible rather than silent — it is not retried, because a capability mismatch is not transient, and the viewer states on screen that what is shown is stored text rather than a live transcription — but the correct fix is configuration: set `MODEL_NAME_VISION` to the vision-capable model.

Provenance follows the split. A document records the model that read it and a message records the model that translated it; naming one for the other would put a false engine in the gold copy. `describe()` prints both at pod startup.

A gateway publishing a single model needs no extra configuration — leave `MODEL_NAME_VISION` unset and both paths use `MODEL_NAME`.

Each pod settles this against the gateway itself at startup, once. Every pod calls `GET /v1/models` once and prints what it found, so the two failures that are otherwise invisible until an officer meets them are visible in the pod log instead:

```
model gateway on – gw.enclave.local:gpt-oss-5.2 – vision: claude-sonnet-4-5
gateway publishes 2: gpt-oss-5.2, claude-sonnet-4-5
```

WARNING: MODEL_NAME "gpt-4o-mini" is not published – calls will 404 and read like an outage
 MODEL_NAME_VISION is unset, so scans go to "gpt-4o-mini". Candidates that look vision-capable: claude-sonnet-4-5

A gateway that will not answer `/models` is not a reason to refuse to start; the pod says so and carries on.

Capability is established by evidence, not by asking. `/models` returns ids and no capability metadata, and a model asked about itself will answer from training rather than from what this gateway serves — a text-only or quantised build carries the same name, and enclave gateways publish internal ids no model has seen. So each pod sends **one 8x8 test image**: whichever model accepts it can read a scan. That runs once at startup and never again, and the result is fixed for the life of the process, so every document a pod handles goes to the same engine and the gold copy's provenance is stable.

- `MODEL_NAME_OCR` **set** — one probe confirms it, and the log warns if that model refuses images rather than letting every document fall back silently.
- `MODEL_NAME_OCR` **unset** on a multi-model gateway — the published models are probed in turn and the first that accepts an image is used, with the choice named in the log.

Pinning it in the Secret is still preferred: it removes the probe's ordering from the answer entirely, which is the same reproducibility argument as the pinned `temperature: 0`.

6.2 WHAT THIS DEPENDS ON FROM THE GATEWAY — AND WHAT IT DELIBERATELY DOES NOT

The whole surface is **two endpoints, and no more**:

POST	<code>{MODEL_ENDPOINT}/chat/completions</code>	every translation, every document
GET	<code>{MODEL_ENDPOINT}/models</code>	startup catalogue check only

Every call is a **single non-streaming POST**. Nothing here uses the Responses API, and nothing opens a stream — no `stream: true`, no server-sent events, no incremental rendering. An officer waits for a whole translation, which is the right unit anyway: a verdict is passed on a finished rendering, not on a half-arrived one.

That is a compatibility position, not an accident. **The enclave gateway supports chat completions and not Responses or streaming** — which is why the Codex CLI cannot be used inside, and why this application can. If the gateway later adds Responses, nothing here needs to change; a deployment that only ever has chat completions is fully served.

The `/models` call is a convenience, not a dependency: a gateway that does not implement it produces one line in the pod log saying names could not be verified, and everything else works.

6.2.1 PARAMETER DIVERGENCE WITHIN CHAT COMPLETIONS IS NEGOTIATED, NOT CONFIGURED

The chat-completions wire format changed: older models accept `max_tokens`, the GPT-5 family rejects it and requires `max_completion_tokens`. Against a gateway publishing GPT-OSS 5.2 this is not a detail — every call returns

```
400 Unsupported parameter: 'max_tokens' is not supported with this model.
```

and every translation and every document falls back to its fixture while the pod reports a healthy gateway. It was found by running the real provider against a real endpoint; nothing in the test suite could have caught it, because a stub accepts whatever it is sent.

The provider sends `max_tokens`, and on that specific 400 switches to `max_completion_tokens` and remembers the choice for the life of the process — one 400 per pod, not per call.

`MODEL_TOKEN_PARAM` overrides it if a gateway ever needs pinning.

temperature diverges the same way, and costs something. Reasoning-family models reject any non-default value:

```
400 Unsupported value: 'temperature' does not support 0 with this model.
    Only the default (1) value is supported.
```

Verified live: `o4-mini` and `gpt-5-nano` refuse it, `gpt-5.2` accepts it — so which behaviour the enclave gets depends on which model it publishes. The provider drops the parameter on that 400 and continues, because a non-deterministic translation beats no translation and a silent fixture.

But `temperature: 0` was chosen so the same document translated twice produces the same gold copy, and dropping it gives that up. The pod log says so explicitly rather than absorbing it:

```
[model-gateway] this model refuses a fixed temperature; omitting it for the rest of
this
process. NOTE: output is no longer deterministic, so re-running a translation may not
reproduce an earlier gold copy verbatim.
```

If reproducibility matters more than the model choice, publish one that accepts a fixed temperature.

`MODEL_TEMPERATURE=omit` skips the probe where the answer is already known.

Both are settled at pod start, not on the first request. A divergence is discovered from a 400, and whichever call triggers it pays a wasted round trip. Left to happen naturally that call is the first officer's first translation, with them watching, so the preflight spends one trivial request to get it out of the way. The pod log states the outcome:

```
wire format negotiated: max_completion_tokens · no temperature (output is not deterministic)
```

and says nothing at all when the gateway needed neither. The result holds for the life of the process — one negotiation per pod, never per call.

No configuration is required for either divergence.

6.3 WHAT THE GATEWAY MUST RETURN

The wire format is OpenAI-compatible chat completions, but the *content* of the reply is a contract of its own. A gateway that speaks the protocol and answers in a different shape will fail in ways that look like bad translation rather than bad integration, so the two shapes are stated here.

`providers/model-gateway.js` is the implementation.

Translation — the assistant message content is the translation itself, as **plain text**. No JSON envelope, no preamble, no commentary; the system prompt instructs the model to return only the translation, and the reply is used verbatim. A gateway that helpfully wraps it (`{"translation": "..."}`) or prefixes it (`Here is the translation:`) will put that wrapper into the officer's gold copy.

OCR / document exploitation — strict JSON, one object per transcribed line:

```
{ "lines": [ { "src": "...", "en": "..." } ], "englishGloss": "..." }
```

The pairing is inside the object deliberately, and a reimplementer must keep it there. The obvious alternative — a `lines` array beside a parallel `en` array — cannot be made safe. A model transcribing a form merges wrapped lines and splits table cells, so the two arrays come back with different lengths; every row after the divergence then renders the wrong English against the wrong source, and nothing on screen looks broken. An officer would certify a pairing the machine never made. One object per line makes that mismatch impossible to express rather than merely unlikely.

Two tolerances are built in and should be preserved:

- **Bare strings are accepted.** A gateway on an older prompt returning `"lines": ["..."]` still transcribes correctly; those lines simply carry no `en`, and the viewer falls back to the whole-document `englishGloss` rather than rendering an empty column beside every line.
- **Pairing is all-or-nothing per page.** A partly-paired page would leave gaps that read as *"this line was not translated"* when the truth is that the service did not supply it.

`tests/node/ocr-fixtures.test.js` holds the demonstration fixtures to the same guarantee, so the mock cannot promise something a live gateway is not asked to produce.

7. Deployment

Two equivalent descriptions, and CI fails if they diverge:

- **kustomize** — `deploy/k8s/` with overlays for TLS.
- **Helm** — `deploy/chart/`, for ArgoCD.

`scripts/compare-chart-kustomize.py` renders both and compares every behaviour-deciding field. It is a build gate, not a convention: the two cannot drift silently.

Posture. All six pods: non-root, read-only root filesystem, dropped capabilities, `seccompProfile: RuntimeDefault`, no privilege escalation — the Restricted Pod Security Standard. Resource limits are 250m CPU / 256Mi memory each, inside the TDD's 0.5 vCPU / 512 MB ceiling. A `NetworkPolicy` restricts service traffic to the SPA pod. Probes are scheme-aware, which matters: leaving them on HTTP after enabling TLS produces an infinite restart loop that looks like a crash.

Restricted-network transfer (`deploy/AIRGAP.md`): fonts vendored so the SPA fetches nothing at runtime; base images digest-pinned; Dockerfiles parameterized for in-enclave bases and mirrors; a registry-prefix transform; an optional enclave-CA mount; and an architecture gate that refuses to package images built for the wrong CPU — an arm64 bundle loads into an x86_64 cluster without complaint and then every pod dies with an exec format error.

Document scans are served as **rasters**, not SVG. The SVG sources name Arabic and CJK fonts that exist on macOS and not on a typical Linux workstation; rasterizing removes the enclave's dependency on having them.

8. Interfaces

8.1 EXTERNAL

Interface	Direction	Protocol	Notes
Browser → <code>imax-spa</code>	in	HTTPS	TLS terminated in-pod (§3)
Authenticating front → <code>imax-spa</code>	in	headers on each request	§4
<code>imax-spa</code> → enrichment services	out	HTTP, in-cluster	<code>UPSTREAM_*</code>
Enrichment services → model gateway	out	HTTPS	Optional; fixtures otherwise (§6)
initContainer → Secrets Manager	out	HTTPS, SigV4	Once at pod start (§3)

8.2 API SURFACE

All under `/api`, all JSON, all schema-pinned:

```

GET  /api/search/threads      thread list + facet counts
GET  /api/search/threads/:threadId/messages  thread message stream
GET  /api/search/messages    corpus-wide search
GET  /api/search/groups      geo-fence and watchlist groups
POST /api/translate           message → English
POST /api/entities           message → typed entities
POST /api/summarize          thread → summary
POST /api/ocr                attachment → blocks[{text,en,bbox}] +
gloss
GET  /api/whoami             resolved caller identity + diagnostics

```

8.3 CONFIGURATION

Everything environment-specific is a ConfigMap or Secret value; nothing enclave-specific is committed, and `scripts/preflight-airgap.sh` fails the build if it ever is.

Object	Carries
<code>imax-auth</code> (ConfigMap)	<code>AUTH_MODE</code> , header and claim names, <code>AUTH_REQUIRED_GROUPS</code>
<code>imax-model-gateway</code> (ConfigMap)	<code>MODEL_ENDPOINT</code> , <code>MODEL_NAME</code> , <code>MODEL_NAME_OCR</code> , <code>MODEL_TIMEOUT_MS</code> — fallback values
<code>imax-model-gateway</code> (Secret)	<code>MODEL_ENDPOINT</code> , <code>MODEL_API_KEY</code> , <code>MODEL_NAME</code> , <code>MODEL_NAME_OCR</code> — the enclave's source of truth; overrides the ConfigMap
<code>imax-tls-config</code> (ConfigMap)	TLS file paths, client-auth mode, upstream scheme
<code>imax-tls-source</code> (ConfigMap)	AWS region, Secrets Manager endpoint, secret names
<code>imax</code> (ServiceAccount)	IRSA role ARN annotation

9. Rotation, scaling and failure

- **Certificate rotation** — the `initContainer` fetches once at pod start, so a rotated certificate is picked up by `kubectl rollout restart`. Continuous refresh would need External Secrets Operator or a sidecar; noted, not built.
- **Scaling** — every pod is stateless, so replica count is a free variable.
- **Failure modes** and their triage lines are tabulated in `deploy/AIRGAP.md`. The ones worth knowing here: missing certificate material holds the pod in `Init:Error` rather than letting it serve plaintext on a port the cluster believes is TLS; a missing model gateway falls back to fixtures and says so at startup; a claim-name mismatch 401s every request and is diagnosed through `/api/whoami`.

10. What the Government still has to supply

Nothing in this list is a defect; each is a value that cannot be known outside the enclave. They are gathered here so that standing the system up is a checklist rather than an investigation.

#	Value	Consequence if wrong
1	Secrets Manager endpoint and whether a VPC endpoint exists in the target VPC	<code>ENOTFOUND</code> at pod start; no route to the API
2	Certificate secret names, and the role granting <code>secretsmanager:GetSecretValue</code>	<code>AccessDenied</code> or a named not-found failure at pod start
3	Whether the certificate's SANs cover all six in-cluster service names	502 after enabling TLS — the likeliest failure
4	The STS claim names a real token carries	401 on every request; <code>/api/whoami</code> reports the actual names
5	<code>MODEL_ENDPOINT</code> , <code>MODEL_NAME</code> , <code>MODEL_API_KEY</code>	Silent fixture fallback, visible in the startup log
5a	<code>MODEL_NAME_OCR</code> — which of the gateway's models can read an image	OCR fails on every page and every document falls back to stored text; the viewer says so on screen, and the startup log names both models
6	In-enclave base images and npm mirror	Image build fails in the enclave
7	Whether the enclave's model gateway returns the reply shapes in §6.3 — plain-text translations, and OCR as one <code>{src, en}</code> object per line	Configured and reachable but wrong-shaped: translations carry a JSON wrapper into the gold copy, or documents lose their per-line English and fall back to the whole-document gloss

Items 1–4 are ConfigMap edits. Item 5 is a ConfigMap and a Secret. Item 6 is documented in `deploy/AIRGAP.md` and parameterized in both Dockerfiles. Item 7 is a prompt-level contract stated in §6.3 and enforced on the fixtures by `tests/node/ocr-fixtures.test.js`; if the enclave's gateway cannot be prompted to that shape, the adapter is `providers/model-gateway.js` and nothing above it changes. **None requires a change to the application**, which is the property Article I(b) is asking about.